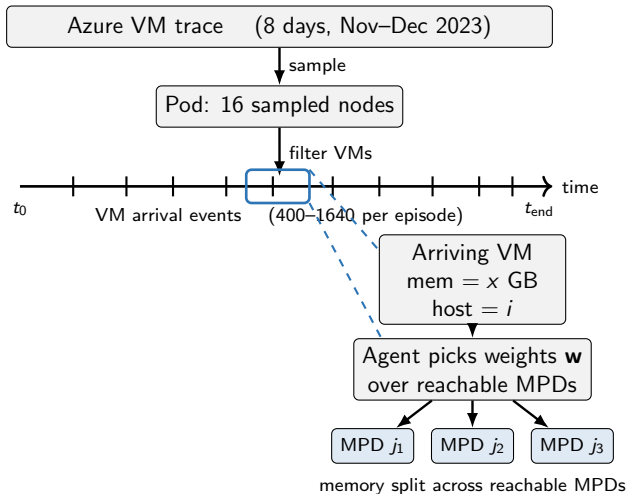


Octopus CXL Memory Pooling: Data Augmentation, Training, and Reward Redesign

Pulak Mehrotra

March 2026

Each training episode replays one pod's VM arrivals as a sequence of allocation decisions.



The training data is ten Azure VM traces replayed as episode streams.

Trace corpus	Episode structure
▶ 10 production Azure VM traces; 8-day window (Nov–Dec 2023) ▶ Locations: Amsterdam, Bangalore, Dublin, London, Singapore, and others ▶ VM counts range from 23k (London) to 271k (Louisville) per trace	▶ Per episode: sample a <i>pod</i> of 16 nodes from one trace ▶ Each VM arrival becomes one allocation event: (tick, host, memMB, dealloc_tick) ▶ Episode length: 400–1640 events depending on trace and pod

The environment reads one memory field per VM; data quality artifacts are filtered before training.

Key data signal: `rss[1]`

The environment reads one field per VM: static memory allocation in MB. Values cluster at powers of two (512, 1024, 2048 MB, ...) because Azure VMs are provisioned in standard SKU sizes.

Time-series utilisation fields (`avg_mem_ts`, `max_mem_ts`) exist in the trace but are not yet consumed by the environment.

Data quality

- ▶ 14–19 % of VMs carry a sentinel `end_time` (`datetime(2020,1,1)`), giving a negative computed lifetime. All are caught and removed by the HOTFIX filter.
- ▶ 4–14 % have zero lifetime (`start = end`).
- ▶ At the default scale, **81 %** of VMs survive into the episode event stream.

Episode-level augmentation generates diverse operating regimes without breaking SKU structure.

Why augmentation is necessary

With one trace and one fixed pod assignment, the policy memorizes a single datacenter's workload pattern and fails to generalize. Augmentation is the standard remedy for train-eval distribution mismatch [1, 2].

Why episode-level, not per-VM

`rss[1]` is a discrete SKU variable: values cluster at powers of two (512, 1024, 2048 MB, ...). Per-VM scaling creates synthetic memory sizes that do not exist in production.

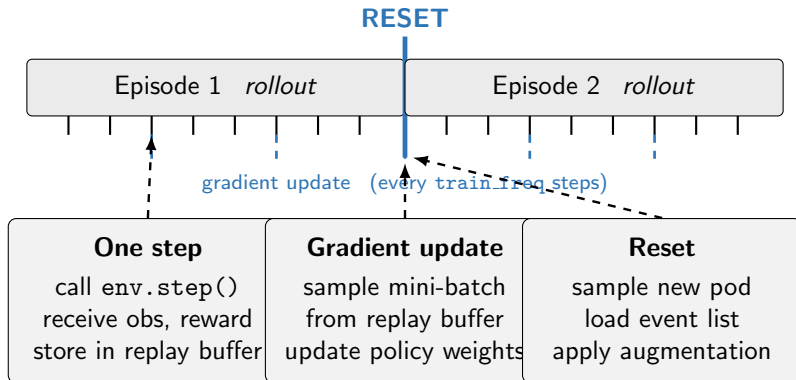
The correct unit is the *episode*: one scale factor s applied uniformly to all VMs, preserving relative sizes. This is the same principle as domain randomization [1]: randomize environment conditions, not individual samples.

Six augmentation knobs span trace diversity, memory scale, and topology perturbation.

Knob	Range / values	Priority
Multi-trace sampling	7 training traces (7/1/2 split)	P0 (always on)
Pod seed randomization	$[0, 2^{31})$ per episode	P0 (always on)
Memory scale s	$\mathcal{U}_{\Delta}(0.7, 1.0, 1.2)$	P0 (always on)
Link failures	$\{0, 0.01, 0.02, 0.03, 0.05\}$	P1 (ablation)
Per-VM noise	$\sigma \leq 0.03$ (lognormal)	P1 (ablation)
Arrival jitter	± 1 tick	P1 (ablation)
Lifetime perturbation	fraction ≤ 0.05	P2 (optional)

Train / validation / test split. 7 traces are used for training (with full augmentation), 1 for validation (no augmentation), and 2 for final held-out test (no augmentation, used once). Augmentation is *never* applied during evaluation so that metrics reflect real trace distributions.

A rollout collects steps within one episode; a reset ends the episode and starts the next.



The previous training setup had three compounding bottlenecks.

Throughput bottlenecks

- ▶ **Serial rollout.**

DummyVecEnv($N = 1$); ~ 46 steps/sec; GPU utilization $< 10\%$.

- ▶ **Slow reset.** Each episode start runs an $O(|\text{VMs}|)$ filter-and-sort pass; ~ 120 ms.

- ▶ **Few gradient updates.** 1M steps at `train_freq=16` yields only $\sim 62k$ policy updates. Standard vs. fast config: no measurable difference in final policy quality.

Policy failure: $20\times$ MPD imbalance

- ▶ SAC collapsed all load onto one or two MPDs; max/min load ratio $\sim 20\times$.

- ▶ Greedy (naïve first-fit) outperformed SAC on held-out evaluation.

- ▶ Root cause: state and reward design, not training budget. The policy converged early to a degenerate solution because the reward gave signal only on the single worst MPD.

Parallel environments and a precomputed event cache deliver a 26× throughput gain.

Fix 1: SubprocVecEnv ($N = 32$)
 N workers run in parallel; observations batched into one GPU kernel per step.

Fix 2: Precomputed event cache
Pod event lists built offline.
`env.reset()` becomes $O(1)$.
Reset: 120 ms $\rightarrow$ 7 ms.

Configuration	Steps / sec	Speedup
Baseline DummyVecEnv ($N = 1$)	46	1×
SubprocVecEnv $N = 4$	191	4.1×
SubprocVecEnv $N = 8$	369	8.0×
SubprocVecEnv $N = 16$	657	14.2×
SubprocVecEnv $N = 32$	1221	26.5×

Key finding. Increasing `train_freq` and `batch_size` alone gave no gain ($\sim 1\times$). The bottleneck was Python/CPU serial throughput, not GPU kernel size. Parallelism via SubprocVecEnv scales near-linearly up to $N = 32$.

The original reward only sees one MPD per step and ignores time.

minimize: **increase in peak load on the single worst MPD this step** + penalize: **pod-wide load variance**

- ▶ **Only one MPD generates signal.** The reward only changes when the *worst* MPD changes. All others are invisible — we get zero feedback no matter how badly we pack them.
- ▶ **Load is treated as permanent.** An MPD at 100 GB emptying in 30 minutes looks identical to one staying full for 48 hours. The policy cannot distinguish them.
- ▶ **Minimizing the next step $\neq$ minimizing the end-of-trace peak.** A placement that looks good now can become a bottleneck once short-lived VMs leave and long-lived ones dominate.
- ▶ **Result: greedy beats SAC.** SAC collapsed all load onto one MPD; max/min imbalance $\sim 20\times$. Greedy outperformed it on held-out traces.

Key idea: ask how loaded each MPD will be after near-term departures.

**projected load = current load – memory leaving within
~12–25 hrs**

What changes

- ▶ A VM departing in 10 minutes contributes almost *nothing* to projected load.
- ▶ A VM with a 3-day lifetime counts in full (*sticky load*).
- ▶ Two MPDs at identical current load can have very different projected loads.

Why this fixes the problems

- ▶ Policy can now prefer full-but-draining over full-and-sticky.
- ▶ Placements scored against durable load — better proxy for end-of-trace peak.
- ▶ State expands 20 → 50 dimensions to expose this decomposition per MPD slot.

Two reward variants trade off local vs. global awareness.

Reward A	Reward B
Optimize: minimize the worst projected load among the MPDs this host can directly reach. Every reachable MPD now contributes signal every step — not just the pod-wide worst. Dense, local, interpretable.	Optimize: same as A, plus a small penalty for hot MPDs this host <i>cannot</i> reach. Hosts share MPD neighborhoods. A host that overloads a shared MPD makes placement harder for its neighbors, even though it cannot observe their decisions directly. Ablation variant: penalty weight $\lambda \in [0.1, 0.3]$.

Both variants are implemented and ablation results will determine which to use for the main training run.

References I

- [1] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IROS*, 2017.
- [2] M. Laskin, K. Lee, A. Stooke, L. Pinto, P. Abbeel, A. Srinivas. Reinforcement learning with augmented data. In *NeurIPS*, 2020.
- [3] R. Raileanu, M. Goldstein, D. Yarats, I. Kostrikov, R. Fergus. Automatic data augmentation for generalization in reinforcement learning. In *NeurIPS*, 2021.
- [4] H. Li, D. S. Berger, L. Hsu, et al. Pond: CXL-Based memory pooling systems for cloud platforms. In *ASPLOS*, 2023.
- [5] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, R. Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *SOSP*, 2017.